

Final Project Report for Image Denoising Using an Autoencoder

CS5300

Kai Watanabe

- **Introduction**

- Goal

- Train an autoencoder that receives a noisy input image and reconstructs a clean output image.
 - Measure the performance of the autoencoder visually, quantitatively(MSE/PSNR/SSIM)

- **Model Architecture**

- The network has three main parts: an Encoder, a Latent Vector, and a Decoder

- Encoder

- It compresses the 28×28 input image into a small size image.
 - In this project, the encoder consists of several convolutional layers followed by Batch Normalization and ReLU activation
 - Feature extraction, downsampling, noise reduction.

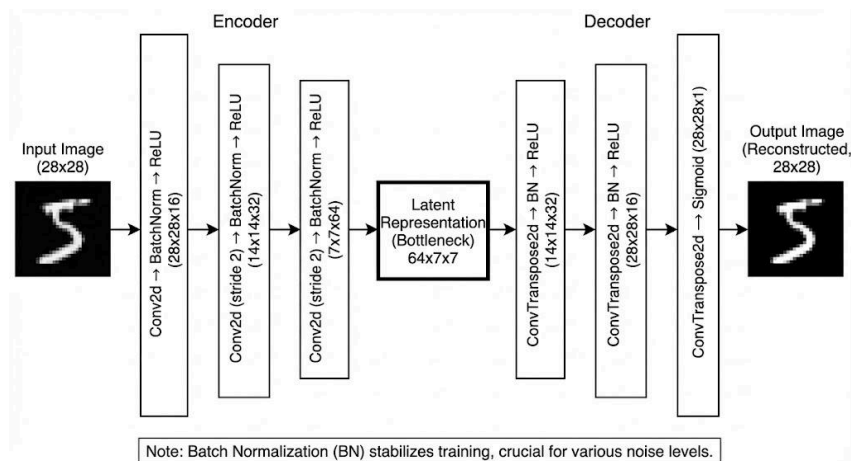
- Latent Vector

- Small, dense internal representation produced by the encoder.

- Decoder

- It takes the latent vector and reconstructs the original image from it.
 - In this project, it uses transposed convolutions, Batch Normalization, ReLU, and a final sigmoid function.
 - Upsampling, reconstruction, batch normalization, sigmoid activation

- Batch Normalization is applied to stabilize the training process and help the model learn effectively across different noise levels.



- **Implementation**

- Environment and Libraries (run on T4 GPU)
 - PyTorch - for the neural network and optimization
 - torchvision - for loading the MNIST data
 - Matplotlib - for visualization
 - Scikit-image - for computing PSNR and SSIM

- **Dataset**

- MNIST dataset for training and testing (grayscale 28 * 28)

- **Training**

- Loss function: Mean Squared Error (MSE)
- Optimizer: Adam (0.001)
- Batch size: 128
- Epochs: 20

- **Noise**

- Gaussian noise
 - The function below takes a clean batch x and a noise level σ , adds Gaussian noise with standard deviation σ
 - During training, I fixed the training noise level to $\sigma = 0.3$, while for evaluation $\sigma = 0.1, 0.3, 0.5$

```
def add_gaussian_noise(x: torch.Tensor, sigma: float) -> torch.Tensor:
    if sigma <= 0: return x
    noise = torch.randn_like(x) * sigma
    return torch.clamp(x + noise, 0.0, 1.0)
```

- **Testing:** To test the performance of the autoencoder, I evaluated the trained model on MNIST test images using three different levels:
 - $\sigma = 0.1$
 - $\sigma = 0.3$
 - $\sigma = 0.5$

For each noise level,

1. The model receive noisy images.
2. Outputs denoised images.
3. Collect both visual samples and quantitative metrics.

And this model also outputs 8 random test images were visualized to show:

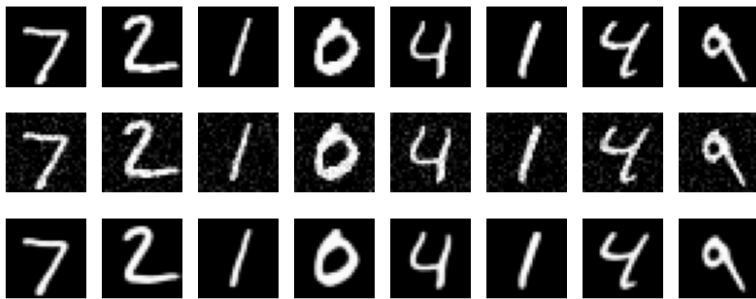
1. Clean image
2. Noisy image
3. Denoised output

- **Result Analysis**

- I evaluated the model with MSE, PSNR, and SSIM
 - MSE (Mean Squared Error) measures the raw numerical difference or error at the pixel level
 - Ideal value: Closer to 0.
 - PSNR (Signal-To-Noise Ratio) measures the quality of the image based on the error level. It indicates how much stronger the original image signal is compared to the distortion.
 - Ideal value: Higher is better.
 - SSIM (Structural Similarity Index) measures the structural quality
 - Ideal value: Closer to 1,

Sigma	MSE	PSNR (dB)	SSIM
0.1	0.0019	27.64	0.9722
0.3	0.0043	23.99	0.9413
0.5	0.0186	17.49	0.6886

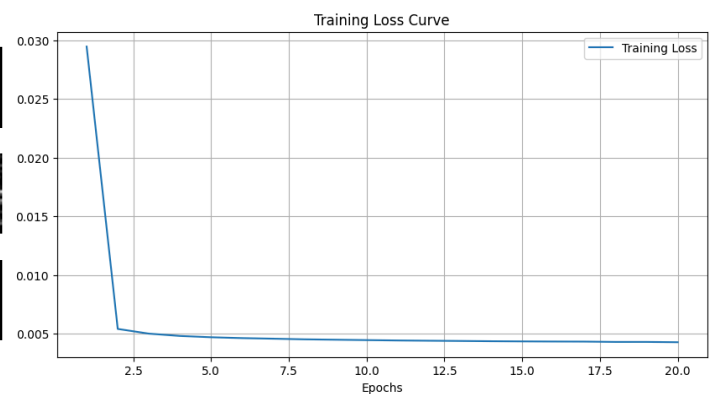
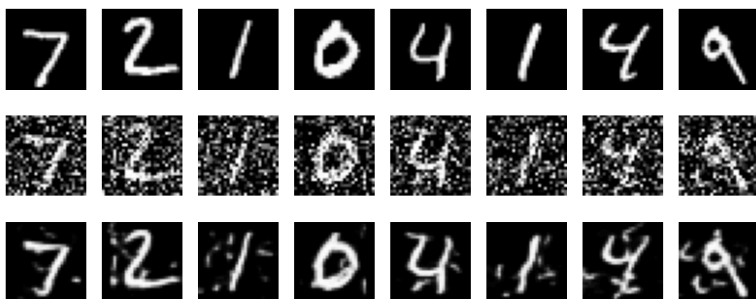
Visual Results for Sigma = 0.1



Visual Results for Sigma = 0.3



Visual Results for Sigma = 0.5



- **Observations**

- At $\sigma = 0.1$, the model reconstructed the images near-perfect.
- At $\sigma = 0.3$, the model still reconstructed the images well.

- At $\sigma = 0.5$, the model was able to reconstruct images, but it still has the noise that it couldn't remove.
- The training loss decrease sharply during the first few epochs and then gradually stabilizes around a very low value. This behavior indicates that the autoencoder quickly learns the basic mapping between noisy and clean images, and then continues to refine the reconstruction.
 - Since the loss does not increase, there's no indication of overfitting during training.

● Goal Reached

- The autoencoder was trained to remove Gaussian noise and produced visually clean images.
- Quantitative metrics shoed the strong performance at moderate noise levels. ($\sigma = 0.1, 0.3$)

● User Guide

- Setup
 - Runtime Environment: Google Colab is recommended. (T4 GPU)
 - Python version 3.x.
 - Libraries
 - Pytorch
 - torchvision
 - numpy
 - matplotlib
 - Scikit-image (skimage is required for PSNR and SSIM)
- How to run the program
 - Running main() will automatically perform the following steps:
 1. Load the MINST dataset
 2. Initialize the autoencoder model
 3. Train the model
 - a. Training noise level: $\sigma = 0.3$
 - b. Epochs: 20
 - c. Loss function: MSE
 - d. Optimizer: Adam(lr= 0.001)
 4. Evaluate the model
 5. Visualize results
 6. Plot the training loss curve

● Reference

- Charu C. Aggarwal, Neural Networks and Deep Learning, 2nd Edition, Chapter 1
- Y. LeCun and C. Cortes, "MNIST handwritten digit database." 1998,
<http://yann.lecun.com/exdb/mnist/>

- PyTorch Documentation: <https://pytorch.org/docs/stable/>
- Autoencoder In Pytorch- Theory & Implementation, Patrick Loeber, Youtube, <https://www.youtube.com/watch?v=zp8clK9yCro&pp=ygUTYXV0b2VuY29kZXIgcHI0b3JjaNgGuAI%3D>
- Building your first Variational Autoencoder with PyTorch, CompBio Skills Seminar UC Berkely, Youtube, https://www.youtube.com/watch?v=IQpP_cH8rrA
- Implementing an Autoencoder in PyTorch, GeeksforGeeks, <https://www.geeksforgeeks.org/deep-learning/implementing-an-autoencoder-in-pytorch/>
- AutoEncoders: Theory + PyTorch Implementation, Syed Hasan, Medium, https://medium.com/@syed_hasan/autoencoders-theory-pytorch-implementation-a2e72f6f7cb7
- Batch Normalization: A Deep Dive For Machine Learning Enthusiasts, Pouya Hallaj, Medium, <https://medium.com/@pouyahallaj/batch-normalization-a-deep-dive-for-machine-learning-enthusiasts-60e4a76d10e8>